ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Master's Thesis Proposal

Secure– and Privacy–aware Object Relational Mappers

Supervisor: Dr. Srđan Krstić, Hoang Nguyen
Professor: Prof. David Basin

1 Prerequisites

Familiarity with databases, SQL, web development and model-driven development.[1]

2 Motivation

2.1 Enforcing fine-grained access control

Enforcing security policies, in particular, fine-grained access control (FGAC), in today's data-centric applications is both critical and challenging. It is critical because of the high economic (e.g., fines) and social (e.g., bad reputation) consequences of failing to enforce them over privacy-sensitive data. As enforcing FGAC policies is not yet well-supported by the current technologies, it is often implemented in an ad-hoc manner, which is error-prone and requires a lot of time.

Nguyen and Clavel [7] propose FGAC model and its characterization for SQL queries. Based on this characterization, they have implemented an enforcement mechanism as a software component called `SQLSI` [8, 2]. `SQLSI` rewrites an SQL `SELECT` query into an SQL stored procedure with authorization checks injected based on the FGAC model. Intuitively, whenever a query is to be executed, instead of executing it directly without any security mechanism in place, the database system executes the corresponding stored-procedure generated by `SQLSI`, which is secure according to the FGAC model.

In practice, typical three-tier system architectures use an Object Relational Mapper (ORM) that automatically maps between the objects in the memory and database tables. Instead of writing SQL queries, developers execute actions (e.g., reading an attribute) on data persistent objects, which ORM translates into the equivalent SQL queries. This hides and simplifies developers' interaction with the database. Recently, Piskor [5] has developed a software component that intercepts these actions and modifies the generated SQL queries to enforce FGAC using SQLSI [2].

2.2 Enforcing purpose-limitation and consent

Data protection regulations, such as the GDPR [4], require IT systems to comply with various privacy requirements. These requirements are particularly challenging to implement as system developers

[1] E.g., see the course on Security Engineering: `https://www.vorlesungen.ethz.ch/Vorlesungsverzeichnis/lerneinheit.view?lerneinheitId=172154&semkez=2023W`

must address them consistently and in a cross-cutting manner. Moreover, without a formal connection between a system's privacy policy and its implementation, arguing for its correctness and evolving it over time becomes extremely difficult.

Recently, Krstić et al. [6] have proposed a model-driven approach with the support for privacy. They define a new privacy model that captures two common privacy requirements: *purpose-limitation* and *consent* and implement two model transformations that generate compliant systems in C# and Python respectively.

The model transformations are implemented in Python directly and support only a fragment of the design modeling elements. Additionally, these transformations generate access control and privacy compliance checks within application layer, interwoven with the application code.

3 Objective

This thesis aims to extend SQLSI [8, 2], named *xtend-SQLSI* in what follows, to support enforcement of purpose-limitation and consent. This extension could potentially (i) enhance the application's performance as the privacy enforcement would be managed by the state-of-the-art database management systems (DBMS), which are optimized for such executions; and (ii) improve further the security and privacy of the application, as (personal) data would never leave the database "unchecked".

Secondly, we aim to refine and integrate this extension with the work by Piskor [5]. Specifically, the extended software component will intercept a relevant actions and modify the auto-generated SQL queries to enforce not only security based on the *FGAC* model but also *purpose-limitation* and *consent* based on the privacy model.

Finally, we plan to conduct case studies to evaluate different aspects of *xtend-SQLSI*.

4 Tasks

We divide the thesis tasks into three work packages (WP).

(WP1): Extend SQLSI to support privacy concepts

1. Understand the proposed FGAC characterization [7], the implementation of `SQLSI` [8], and the proposed model-driven privacy approach [6].
2. As the security model in the FGAC characterization [7] is defined in a different way compared to the one in the model-driven privacy [6]. Investigate and refine (if needed) the proposed model characterization for enforcing FGAC in the FGAC characterization work [7] using the security model definition in model-driven privacy approach [6].
3. Extend the SQLSI approach to support privacy concerns (i.e., purpose-limitation and consent).
   - Extend the model transformation to support generating privacy concepts **in the database schemata** such as *personal data*, *purposes*, *ownership* of data, and *data-subject consent*.
   - (Depends on the previous characterization refinement in item 2) Adjust the model transformation to support generation of **SQL authorization checks** that corresponds to refined model characterization.
   - Extend the model transformation to support rewriting **SQL queries** (i.e., SQL `SELECT` statements) into SQL stored-procedures that are also privacy–aware (with respect to the enforcement of checking purpose limitation and consent).
   - Extend the interface of SQLSI to signal privacy exception(s).
4. (Optional) Investigate and extend the model characterization to support other actions and its SQL relevant statement types (e.g., create action and `INSERT` statements; update action and `UPDATE` statements; and delete action and `DELETE` statements).

(WP2): Integrate the xtend-SQLSI with existing ORM interceptor in [5]

1. Study the selected ORM implementation (`LINQ` [1, 3]), the proposed approach and its prototype component in Piskor's work [5].
2. Extend Piskor's component [5] to support additional privacy enforcement using the SQLSI extended toolset in WP1.
   - In order to use SQLSI, [5] needs to first translate the ORM representation classes into the data model. Extend the validator and the translation to support newly introduced privacy concepts.
   - Extend the component to include relevant information when intercepting persistent actions. In addition to, the information on the current user (and possibly its role), one need to forward the collection of actual purposes.
3. (Optional) Improve current limitations of the prototype component in [5].
   - The stored-procedure generation memoization.
   - Default security model for authentication library.
   - Default privacy model and aspects for registration and authentication purposes.
   - Code analysis to detect SQL queries from the application code.

(WP3): Case study and Evaluation

1. Develop case study applications (ideally from the previous work [6, 8]) using *xtend-SQLSI*.
2. Compare the implementation using *xtend-SQLSI* with existing tools, i.e., the current SQLSI and the prototype in [6], and assess its compatibility and correctness.
3. Perform micro-benchmarking on the implementations of *xtend-SQLSI* to assert its performance overhead and compare this overhead with the previous implementations [6].

5 Deliverables

The following deliverables are due at the end of the project:

Final report The final report should consist of an introduction; a theoretical background section; and sections describing the work packages and their outcome; and a conclusion. The report must be written in English.

Source code Complete development including easily reproducible evaluation.

Presentation At the end of the project, a 20 minute presentation must be given in an Information Security group seminar. It should give an overview and discuss the most important highlights of the work.

References

[1] Language-Integrated Query (LINQ) documentation, 2018. `https://learn.microsoft.com/en-us/dotnet/api/system.linq`.

[2] The SQLSI Tool. `https://github.com/MoDELSVGU/SQLSI`, 2022.

[3] LINQ implementation, March 2023. `https://github.com/dotnet/runtime/tree/main/src/libraries/System.Linq`.

[4] European Parliament and Council. Regulation (EU) 2016/679, 2016. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`.

[5] Karlo Piskor. Secure Object-Relational Mappers, 2023. Bachelor Thesis, Department of Computer Science, ETH Zürich.

[6] Srdan Krstic, Hoang Nguyen, and David A. Basin. Model-driven privacy. *Proc. Priv. Enhancing Technol.*, 2024(1):314–329, 2024.

[7] Hoang Nguyen and Manuel Clavel. Model-based Characterization of fine-grained Access Control Authorization for SQL Queries. *J. Object Technol.*, 19(3):3:1–13, 2020.

[8] Hoang Nguyen and Manuel Clavel. A Model-Driven Approach for Enforcing Fine-Grained Access Control for SQL Queries. *SN Comput. Sci.*, 2(5):370, 2021.